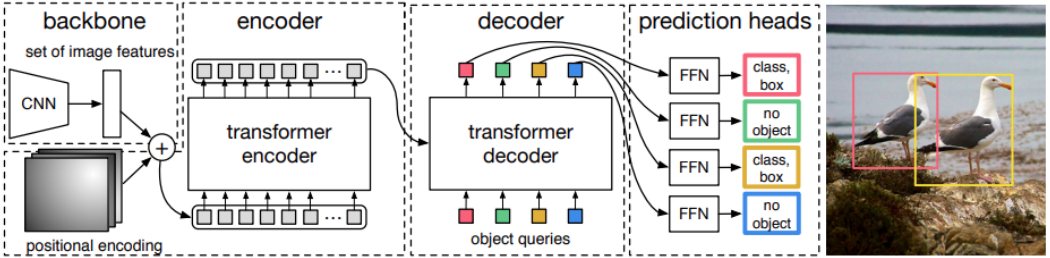
# Transformers

Naeemullah Khan
naeemullah.khan@kaust.edu.sa

جامعة الملك عبدالله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

August 9, 2026




Source: Roboflow — What is DETR? (Carion et al., 2020)

# Table of Contents

# Motivation

Why this topic matters:

- RNNs are sequential and struggle with long-range dependencies.
- Attention lets a model focus on the most relevant parts of the input.
- Transformers replace recurrence with self-attention, making them fully parallelizable.
- They are the backbone of modern NLP and, increasingly, computer vision.
- *"Attention is all you need."* — Vaswani et al., 2017.

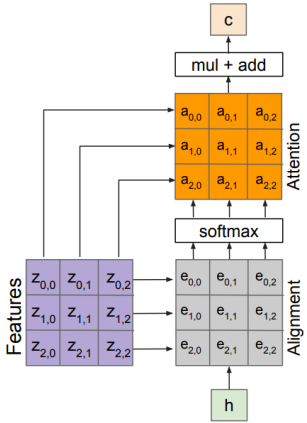
# ⇒ Learning Outcomes

By the end of this session, you will be able to:

- Explain the general attention layer and the role of queries, keys, and values.
- Describe self-attention and why it is permutation equivariant.
- Understand positional encodings, masked attention, and multi-head attention.
- Describe the Transformer block and how blocks stack into a full Transformer.
- Compare Transformers with RNNs in terms of parallelism and long-range modeling.

# Transformers: **General Attention Layer**

# Attention We Just Saw in Image Captioning



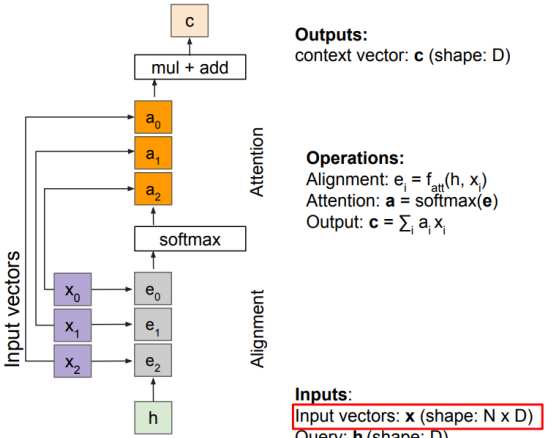

**Outputs:**
context vector: **c** (shape: D)

**Operations:**
Alignment: $e_{i,j} = f_{att}(h, z_{i,j})$
Attention: $\mathbf{a} = \text{softmax}(\mathbf{e})$
Output: $\mathbf{c} = \sum_{i,j} a_{i,j} z_{i,j}$

**Inputs**:
Features: **z** (shape: H x W x D)
Query: **h** (shape: D)

Source: Stanford CS231n (2022), Lecture 11: Attention and Transformers

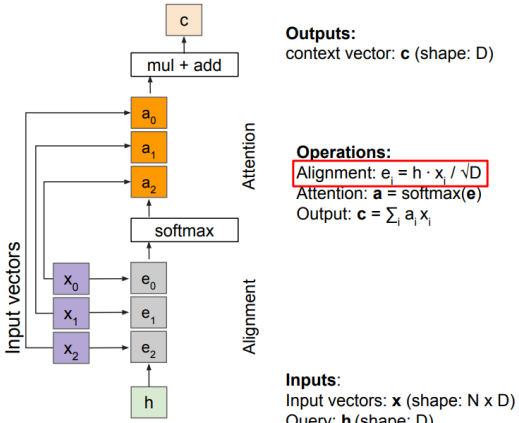
# General Attention Layer




Source: Stanford CS231n (2022), Lecture 11: Attention and Transformers

- The attention operation is permutation invariant.
- It does not depend on the ordering of the features.
- Stretch $H \times W = N$ into $N$ vectors.

# General Attention Layer

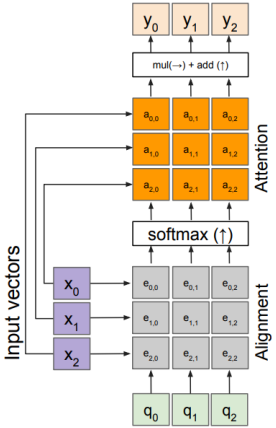



Source: Stanford CS231n (2022), Lecture 11: Attention and Transformers

Change $f_{att}(.)$ to a scaled simple dot product:

- Larger dimensions mean more terms in the dot product sum.
- Therefore, the variance of the logits is higher. Large-magnitude vectors will produce much higher logits.
- As a result, the post-softmax distribution has lower entropy, assuming logits are IID.
- Ultimately, these large-magnitude vectors will cause softmax to peak and assign very little weight to all others.
- Divide by $\sqrt{D}$ to reduce the effect of large-magnitude vectors.

# General Attention Layer




**Outputs:**
context vectors: **y** (shape: D)

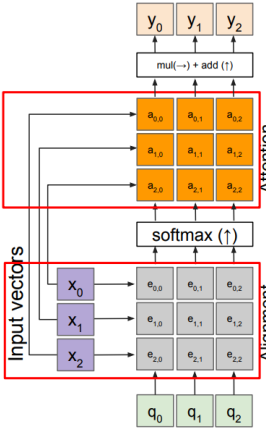
**Operations:**
Alignment: $e_{i,j} = q_j \cdot x_i / \sqrt{D}$
Attention: **a** = softmax(**e**)
Output: $y_j = \sum_i a_{i,j} x_i$

**Inputs**:
Input vectors: **x** (shape: N x D)
Queries: **q** (shape: M x D)

- We can have multiple query vectors.
- Each query creates a new output context vector.

Source: Stanford CS231n (2022), Lecture 11: Attention and Transformers

# General Attention Layer




**Outputs:**
context vectors: **y** (shape: D)

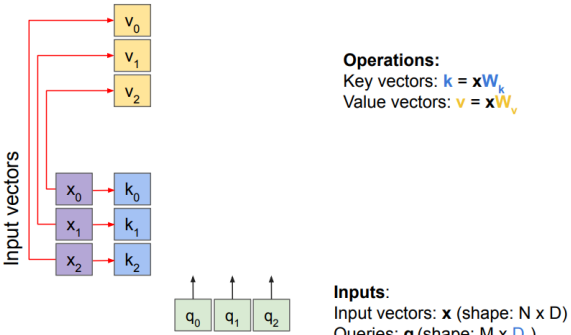
**Operations:**
Alignment: $e_{i,j} = q_i \cdot x_j / \sqrt{D}$
Attention: **a** = softmax(**e**)
Output: $y_j = \sum_i a_{i,j} x_i$

**Inputs**:
Input vectors: **x** (shape: N x D)
Queries: **q** (shape: M x D)

- Notice that the input vectors are used for both the alignment and the attention calculations.

Source: Stanford CS231n (2022), Lecture 11: Attention and Transformers

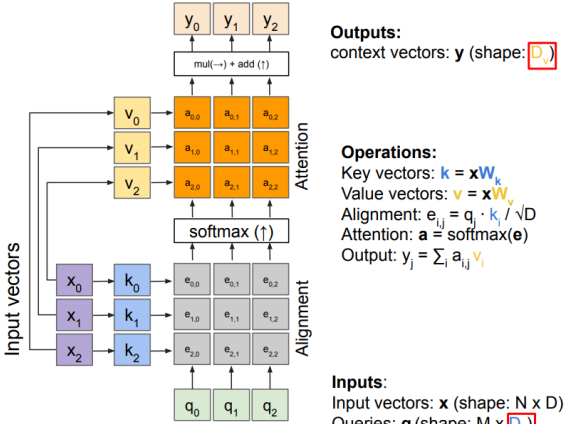
# General Attention Layer




- Notice that the input vectors are used for both the alignment and the attention calculations.
- We can add more expressivity to the layer by adding a different fully connected (FC) layer before each of the two steps.

Source: Stanford CS231n (2022), Lecture 11: Attention and Transformers

# General Attention Layer




- Notice that the input vectors are used for both the alignment and the attention calculations.
- We can add more expressivity to the layer by adding a different fully connected (FC) layer before each of the two steps.
- The input and output dimensions can now change depending on the key and value FC layers.

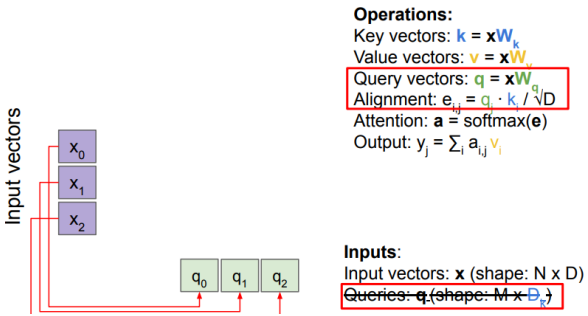
Source: Stanford CS231n (2022), Lecture 11: Attention and Transformers

# Transformers: **Self-Attention Layer**

# Self-Attention Mechanism

- **Self-Attention:** Computes attention within a single sequence.
- **How it works:**
  - Each token attends to every other token.
  - Captures global context, not just local dependencies.

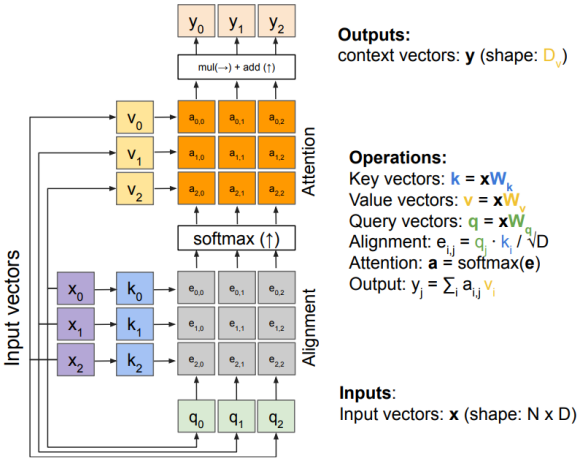
# Self-Attention Layer




Source: Stanford CS231n (2022), Lecture 11: Attention and Transformers

- Recall that the query vector is a function of the input vectors.
- We can calculate the query vectors from the input vectors, thus defining a "self-attention" layer.
- There are no input query vectors anymore.
- Instead, query vectors are calculated using a fully connected (FC) layer.

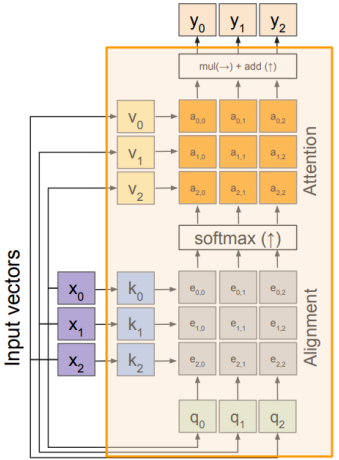
# Self-Attention Layer




Source: Stanford CS231n (2022), Lecture 11: Attention and Transformers

- Recall that the query vector is a function of the input vectors.
- We can calculate the query vectors from the input vectors, thus defining a "self-attention" layer.
- There are no input query vectors anymore.
- Instead, query vectors are calculated using a fully connected (FC) layer.

# Self-Attention Layer

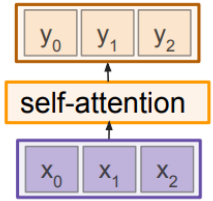



**Outputs:**
context vectors: **y** (shape: $D_v$)

**Operations:**
Key vectors: $k = xW_k$
Value vectors: $v = xW_v$
Query vectors: $q = xW_q$
Alignment: $e_{i,j} = q_i \cdot k_j / \sqrt{D}$
Attention: $a = \text{softmax}(e)$
Output: $y_j = \sum_i a_{i,j} v_i$

**Inputs**:
Input vectors: **x** (shape: N x D)

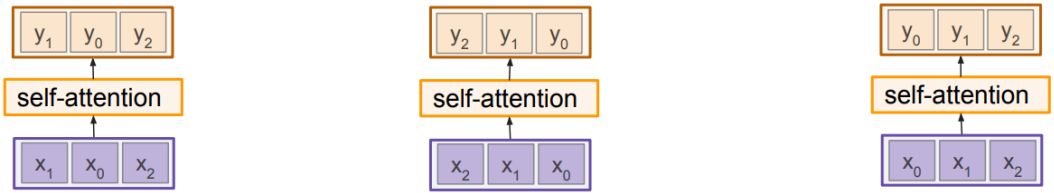
Source: Stanford CS231n (2022), Lecture 11: Attention and Transformers




Source: Stanford CS231n (2022), Lecture 11: Attention and Transformers

# Transformers: **Permutation Equivariance**

# Permutation Equivariance

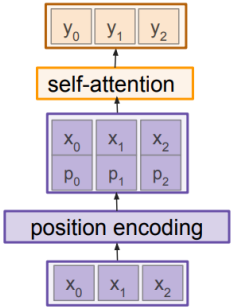



Source: Stanford CS231n (2022), Lecture 11: Attention and Transformers

- Self-Attention Layer is Permutation equivariant
- It doesn't care about the order of the inputs!
- **Problem:** How can we encode ordered sequences like language or spatially ordered image features?

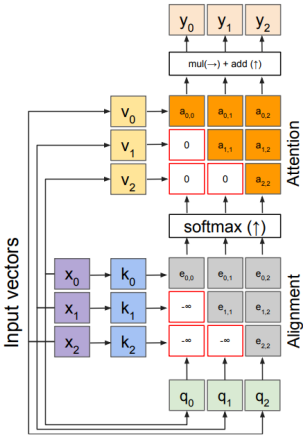
# Positional encoding

- Concatenate/add special positional encoding $p_j$ to each input vector $x_j$
- We use a function pos: $\mathbb{N} \to \mathbb{R}^D$ to process the position $j$ of the vector into a $D$-dimensional vector
- So, $p_j = pos(j)$




Source: Stanford CS231n (2022), Lecture 11: Attention and Transformers

# Masked self-attention layer




**Outputs:**
context vectors: **y** (shape: $D_v$)

**Operations:**
Key vectors: $\mathbf{k} = \mathbf{x}W_k$
Value vectors: $\mathbf{v} = \mathbf{x}W_v$
Query vectors: $\mathbf{q} = \mathbf{x}W_q$
Alignment: $e_{i,j} = q_j \cdot k_i / \sqrt{D}$
Attention: $\mathbf{a} = \text{softmax}(\mathbf{e})$
Output: $y_j = \sum_i a_{i,j} v_i$

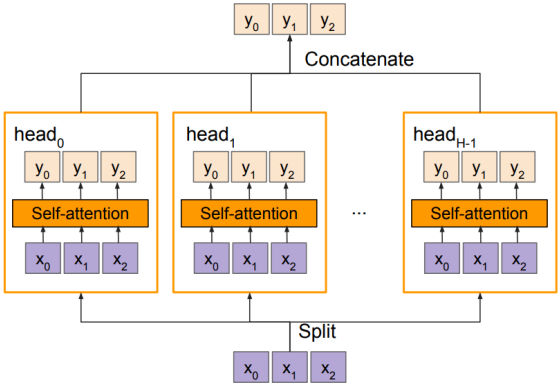
**Inputs**:
Input vectors: **x** (shape: N x D)

- Prevent vectors from looking at future vectors.
- Manually set alignment scores to $-\infty$

Source: Stanford CS231n (2022), Lecture 11: Attention and Transformers

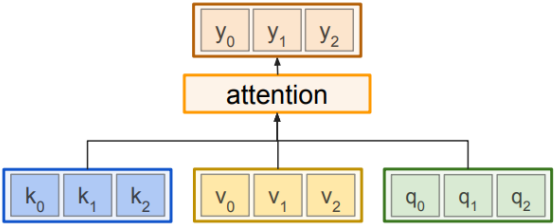
# Multi-head self-attention layer

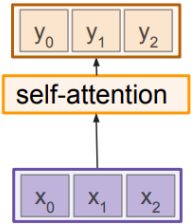
- Multiple self-attention heads in parallel




Source: Stanford CS231n (2022), Lecture 11: Attention and Transformers

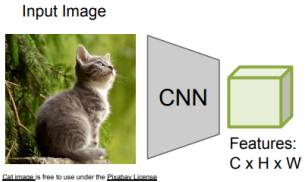
# General attention versus self-attention



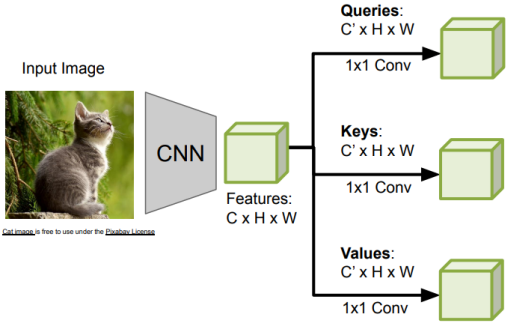

Source: Stanford CS231n (2022), Lecture 11: Attention and Transformers

# Example: CNN with Self-Attention



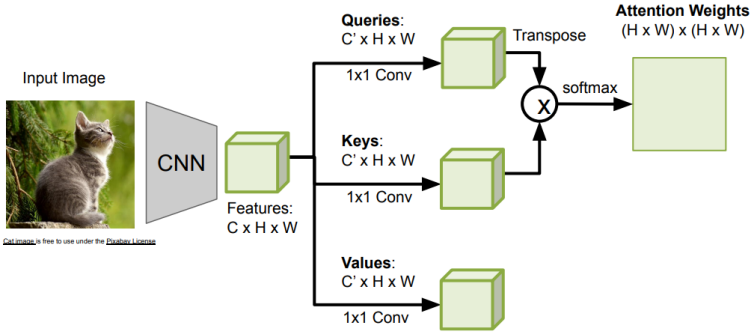

Source: Stanford CS231n (2022), Lecture 11: Attention and Transformers; module after Zhang et al., "Self-Attention Generative Adversarial Networks," ICML 2019

# Example: CNN with Self-Attention



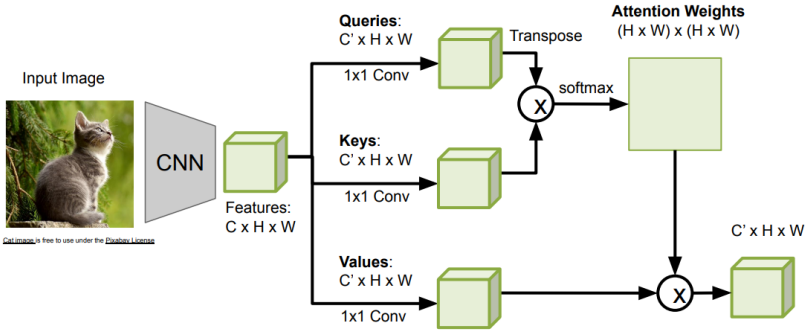

Source: Stanford CS231n (2022), Lecture 11: Attention and Transformers; module after Zhang et al., "Self-Attention Generative Adversarial Networks," ICML 2019

# Example: CNN with Self-Attention



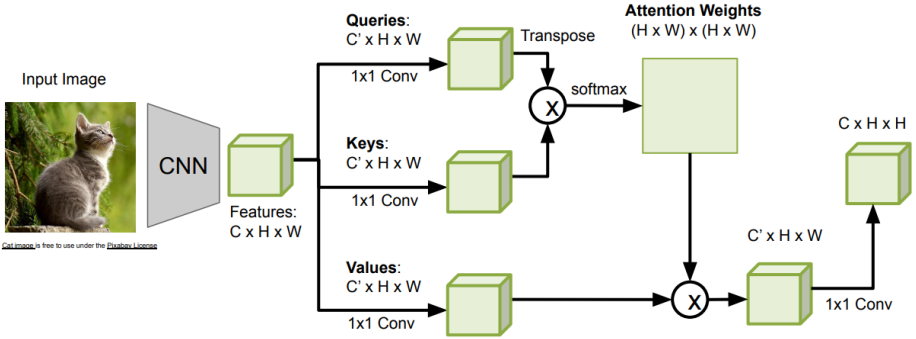

Source: Stanford CS231n (2022), Lecture 11: Attention and Transformers; module after Zhang et al., "Self-Attention Generative Adversarial Networks," ICML 2019

# Example: CNN with Self-Attention



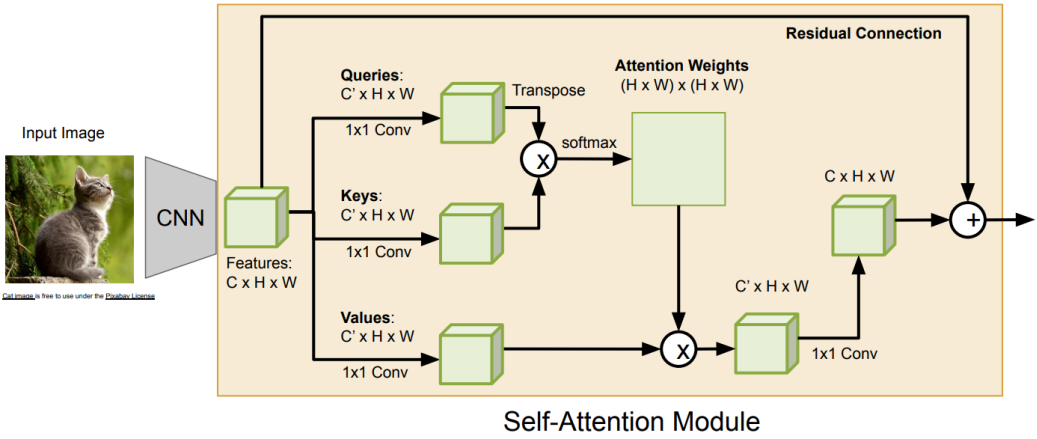

Source: Stanford CS231n (2022), Lecture 11: Attention and Transformers; module after Zhang et al., "Self-Attention Generative Adversarial Networks," ICML 2019

# Example: CNN with Self-Attention




Source: Stanford CS231n (2022), Lecture 11: Attention and Transformers; module after Zhang et al., "Self-Attention Generative Adversarial Networks," ICML 2019

# Example: CNN with Self-Attention




Source: Stanford CS231n (2022), Lecture 11: Attention and Transformers; module after Zhang et al., "Self-Attention Generative Adversarial Networks," ICML 2019

# Transformers: **Transformer Overview**

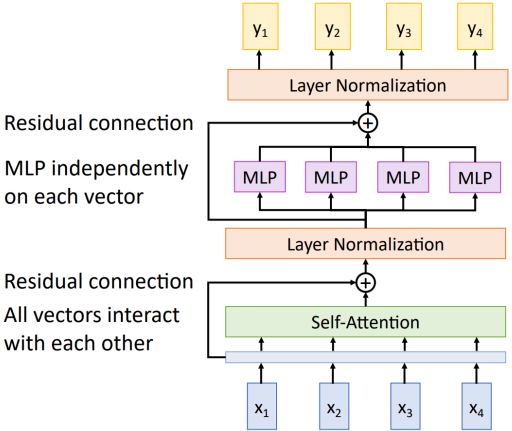
# Attention is all you need

Vaswani et al, NeurIPS 2017

# Transformer Block

- **Input:** Set of vectors **x**
- **Output:** Set of vectors **y**

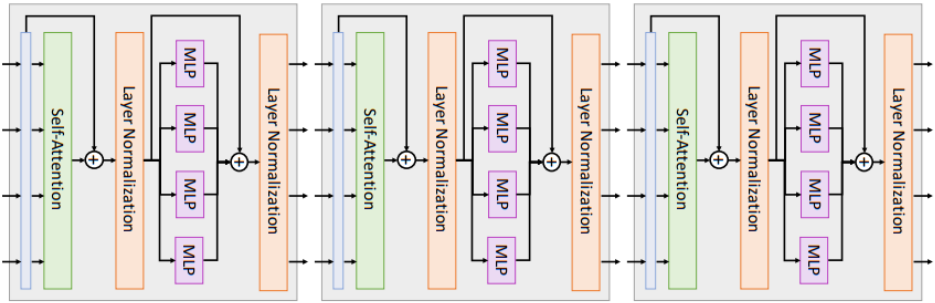
- Self-attention is the only interaction between vectors!
- Layer normalization and MLP operate independently on each vector.
- Highly scalable and highly parallelizable.




Source: UMich EECS 498 (2022), Lecture 17: Attention, J. Johnson

# Transformers

- A Transformer consists of a sequence of transformer blocks.
- Vaswani et al.'s base model used 6 encoder and 6 decoder blocks, $d_{\text{model}} = 512$, and 8 attention heads.




Source: UMich EECS 498 (2022), Lecture 17: Attention, J. Johnson

# Transformers: **RNNs vs Transformers**

# RNNs vs Transformers

| Aspect | RNN / LSTM | Transformer |
|---|---|---|
| Computation | Sequential | Parallel |
| Long-range dependencies | Hard (signal decays) | Direct (any-to-any) |
| Path length between tokens | $O(n)$ | $O(1)$ |
| Complexity per layer | $O(n \cdot d^2)$ | $O(n^2 \cdot d)$ |
| Order information | Built-in (recurrence) | Added (positional encoding) |

- Transformers trade quadratic attention cost for full parallelism and shorter gradient paths.
- This makes them far more scalable on modern hardware, which is the key to large pretrained models.

# Transformers: **Limitations and Future Directions**

# Limitations of Transformers

- **Quadratic complexity**: self-attention scales as $O(n^2)$ in sequence length.
- **Data hungry**: need large datasets or pretraining to perform well.
- **Weak inductive biases**: unlike CNNs/RNNs, no built-in locality or order bias; order must be injected via positional encodings.
- **Memory intensive**: long sequences are expensive to store and compute.

# Future Directions

- **Efficient attention**: Linformer, Performer, Longformer, FlashAttention.
- **Vision**: extend self-attention to images (Vision Transformers, covered later in the course).
- **Multimodality**: unify text, image, and audio in a single architecture (e.g., Flamingo).

# Transformers: **References**

# References


[1] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is All You Need. *Advances in Neural Information Processing Systems*, 30. https://arxiv.org/abs/1706.03762

[2] Bahdanau, D., Cho, K., & Bengio, Y. (2015). Neural Machine Translation by Jointly Learning to Align and Translate. *International Conference on Learning Representations*. https://arxiv.org/abs/1409.0473

[3] Zhang, H., Goodfellow, I., Metaxas, D., & Odena, A. (2019). Self-Attention Generative Adversarial Networks. *International Conference on Machine Learning*, 7354–7363. https://arxiv.org/abs/1805.08318

[4] Fei-Fei Li, Johnson, J., & Yeung, S. Stanford CS231n: Deep Learning for Computer Vision. http://cs231n.stanford.edu/

# References (cont.)

[5] Johnson, J. UMich EECS 498.008/598.008: Deep Learning for Computer Vision. https://web.eecs.umich.edu/~justincj/teaching/eecs498/WI2022/